

teste - 16/17 perguntas e pode responder se a 10/11
1 ou 2 de código

Bioinformatics Module 3 – session 4

Supervised learning II

(Conceitos mais avançados da aprendizagem supervisionada:)

Hyperparameter optimization and feature selection

Neural networks and Deep Learning

ML for protein sequences/ compounds (QSAR)

Feature selection (seleção de atributos)

In many practical cases there is a need/several advantages in **reducing the number of input features** of a model:

- **complexity** of the models increases the number of input features which can cause **overfitting**;
- data and models can be more easily **analysed** and **understood**;
- Eliminations of the redundant or contradictory features can improve the learning process reducing **noise**

The process of choosing the best subset of features of a given set is an **optimization problem** which can become complex given the wide space of demand

Problem similar to model optimization !!

(se houver modelos muito próximos uns dos outros pode ser muito confuso)

ex: tem se 200000 genes e quer se seleccionar de 500 em 500.
normalmente nunca se consegue fazer um problema de otimização com soluções ótimas
normalmente so se consegue avaliar com algumas possíveis hipóteses)

(o machine learning permite trabalhar com multiplas variaveis, mas aqui é como se estivessemos a trabahar com univariaveis, ver coluna a coluna como se processa)

Feature selection: strategy

(ex: se a variavel de output for variavel numerica, da para colocar as variaveis de input e avaliar a correlação)

Filters (são filtros estatísticos que dão para fazer um ranking dos atributos e pode seleccionar o melhor ranking)

Selection of features performed before the learning process, independently of the classifiers

Evaluation of subsets of features performed with statistical measures (e.g. Entropy/gain, etc.)

Wrappers (os proprios modelos no processo de treino fazem features selection já. por ex., ha um metodo de regressao linear e os features proximos de zero não são muito uteis)

Selection of features performed in parallel with the construction of the model

Evaluation of the subsets of features performed by training a model and estimating its error (using the methods studied)

Embedded

Selection of features performed along with the learning process (e.g. linear regression with regularization)

Filters

- **Unsupervised:**

- Calculate a metric from the features values only (e.g. based on their variability)
- Selection can be made by ranking (e.g. percentile) or by absolute value keeping all features with a value below/above a given threshold

- **Supervised:** (comparam o input com o output)

- Based on a metric calculated from the feature values compared to the output value (e.g. mutual information, statistical tests, correlation)
- Can be based on univariate statistical tests depending on the type of feature (e.g. t-test, ANOVA, χ^2 , etc)

(se se tiver uma featurre que mede o mesmo valor, nao serve para nada, por isso podem remover se todas as features com uma variancia menor que X.)

Wrappers

- Heuristics:
 - **Forward selection**: starts with few features (e.g. 1) and adds features until a good performance value is obtained (Heurísticas muito simples e depois introduzem se metricas mais complexas)
 - **Backward selection**: starts with all features and removes iteratively (começa com todas as features e vai removendo)
- Meta-heuristics:
 - **Evolutionary Algorithms** (gastam muitos recursos, muita computação e pode fazer overfitting (ao tentars e otimizar o conjunto de features, pode se formar um subconjunto que funciona bem no fitting mas funciona mal no geral), por isso, evitam-se)
 - **Simulated Annealing**

(hiperparametros - tudo antes de o algoritmo de treino definir, SOMOS NOS QUE DEFINIMOS E NAO MUDA DURANTE O TREINO
parametros de machine learning - o que é aprendido - ex: arvores de decisao, redede pesos neuronal,e tc. o que o algoritmo de treino vai definir. É O QUE O MODELO APRENDE COM O TREINO)

Model selection – optimizing hyperparameters

In order to optimize the construction process, in many cases, a process of **model selection** / searching best (hyper)parameters is carried out

Optimization process by minimizing an error measure on a set of validation examples (not used for the training process) -> required to use error estimation techniques (e.g. resampling, cross-validation)

Objective is to look for models that minimize overfitting and underfitting:
i.e. return models with a good balance between error and complexity

(como é um processo de otimização, exige treinar modelos, valida los, etc. perceber quais as configurações que funcionam melhor.)

Hyperparameter optimization - algorithms

Grid search – considers for each hyperparameter a set of possible alternatives and explores all possible combinations of values of the various parameters, choosing the one that has the best result (pego em cada parametro e vejo todas as combinações possíveis)

Randomized search – similar to the previous but only explores a part of the possible combinations chosen at random (testam se algumas combinações (cada valor para cada parametro testado algumas vezes...aqui tem de haver algum bom senso para nao ter overfitting, etc)

Bayesian optimization – more elaborated methods of stochastic optimization which seek to explore the most promising areas of the search space (nao muito importante)

Other metaheuristics such as Bayesian optimization and evolutionary computation have also been used

In all cases, each combination of hyperparameters is evaluated by training the model with these values and calculating an error estimate, typically using a validation set or cross-validation

Analogy: natural learning

The brain is a highly complex structure, **non linear** and highly **parallel**

It has an ability to organize its neurons, to perform complex tasks

A neuron is 5/6 times slower than a logical port mais lentos que uma porta logica

The brain overcomes this slowness through a **parallel** and highly **connected** structure

The human cortex has 10 billion neurons and 60 trillion synapses!!

(redes neuronais são modelos matematicos que sao usados em machine learning mas nao sao modelos. nao estamos a tentar modelar nem imitar o cerebro, mas sim tentar entender coisas uteis que o cerebro faz e aplicar em modelos matematicos

unidades simples de processamento biologico - neuronio

ha um certo paralelismo devido a rede neuronal)

What are neural networks?

(Artificial) Neural networks are models of machine learning that follow an **analogy with the functioning of the human brain**

A neural network is a **parallel** processor, consisting of simple processing units (neurons)

Knowledge is stored in the **connections** between the neurons

Knowledge is acquired from the environment (data) through a **learning process** (training algorithm) that **adjusts the weights** of the connections

(consegue se por deep learning nas redes neuronais dentro do que é o machine learning.

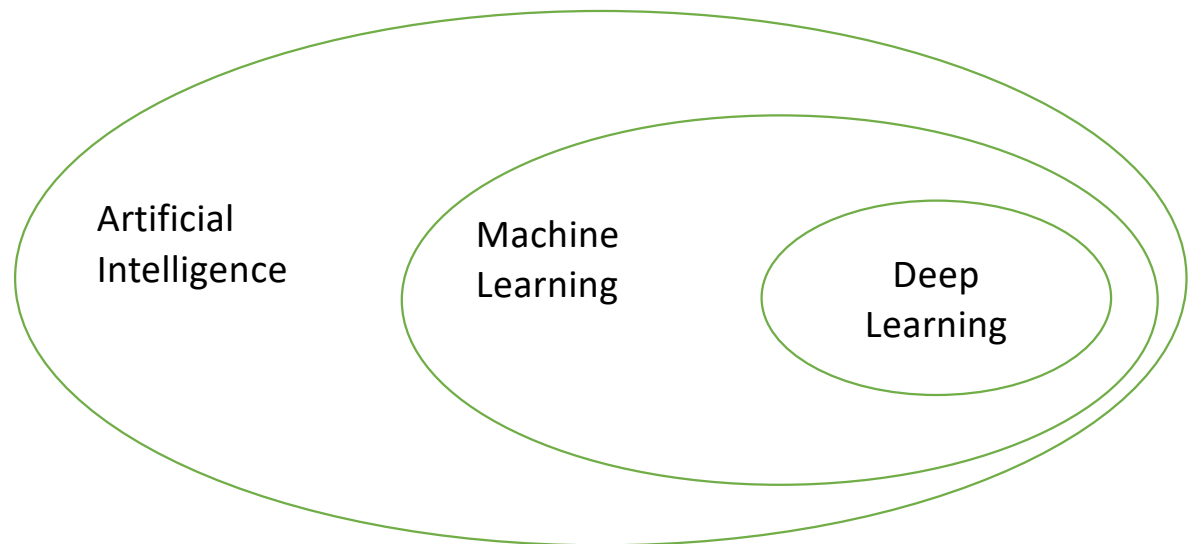
Tem o nome deep porque tem uma complexidade no sentido em que ha muitas camadas no processamento de informação (como legos, varias peças que vao sendo montadas e cada vez maior))

Deep learning: what is it?

(ideia mais recente)

Machine Learning field characterized by a greater complexity of models and the ability to learn representations from input data

Deep learning models consist of successive layers of representations, in a number that is typically high (deep models)



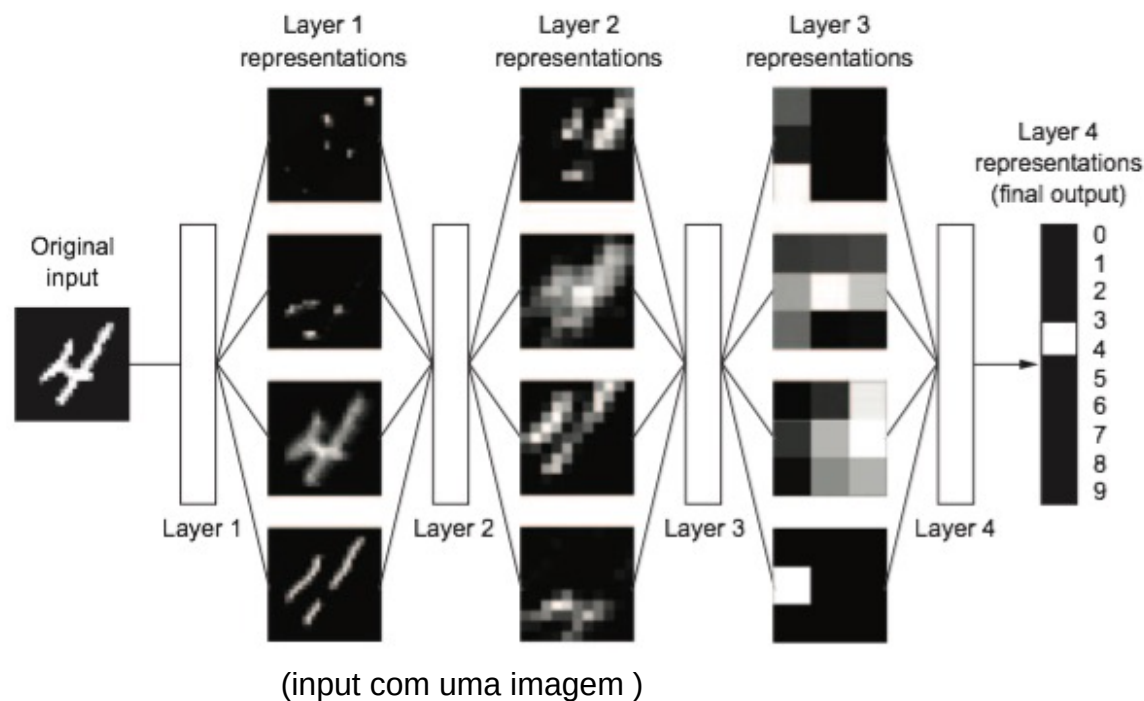
Deep learning: what is it?

Several layers process information by creating distinct and typically more abstract representations of the inputs

In the case of supervised learning, the last layer represents the output of the neural net

Learning by gradient descent methods

(todos os algoritmos de treino de modelos neuronais são baseados em algoritmos de gradiente descendente - IMPORTANTE)



Deep learning: application areas (Aplicações)

DL models have been applied in several fields with good quality results, including:

- Image classification (e.g. ImageNet)
- Recognition of spoken texts
- Transcript of handwritten text
- Automatic text translation
- Answers in natural language / digital assistants
- Games (e.g. Go)
- Retrosynthesis in chemistry
- Classification of protein and DNA sequences

(gpu - parte do pc que refresca o ecran (ex: 50 hz, significa que refresca o ecran 50 vezes))

Deep learning: major factors

Like any technology, DL does not solve all problems and will not always be the best option for any learning task

A determining factor for DL success is the availability of large-scale datasets; for problems with little data, other models can give more consistent results with less computational effort

In terms of hardware, the use of graphics processors (GPU) brings advantages in training DL models by accelerating the process by factors of more than 10x

Also, some technical improvements in relation to "shallow" neural networks

Artificial neurons

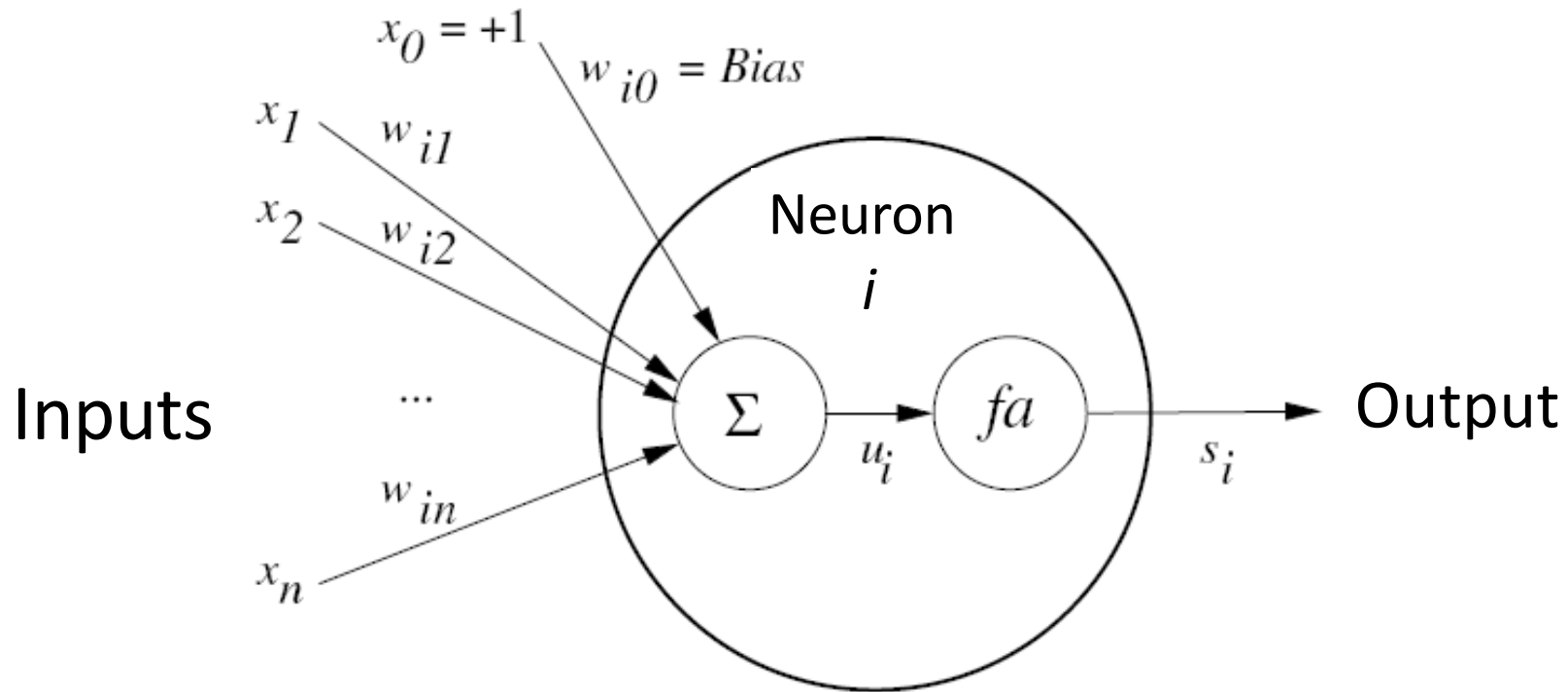
Receive a set of inputs (data or connections)

A **weight** (numerical value) is associated with each connection

Each neuron calculates its **activation** based on the input values and the weights of the connections

The calculated signal is passed on to the output after being filtered by an **activation function**

Structure of a neuron



(multiplicar valores dos inputs (x_1, x_2) por pesos/parametros (w_{i1}, w_{i2})

---> é o caso do modelo de regressao linear (aula 9)

se se aplicar uma função sigmoide)

(neurônio está ligado a outros neurônios (podem ser inputs do dataset ou podem ser coisas que vem de outros neurônios como x_1 e x_2) e multiplica e soma tudo e calcula o somatório de multiplicações. Vai então dar origem a funções de não ativação (sigmoide, tangentes, etc)

há várias variáveis destes neurônios)

Activation functions

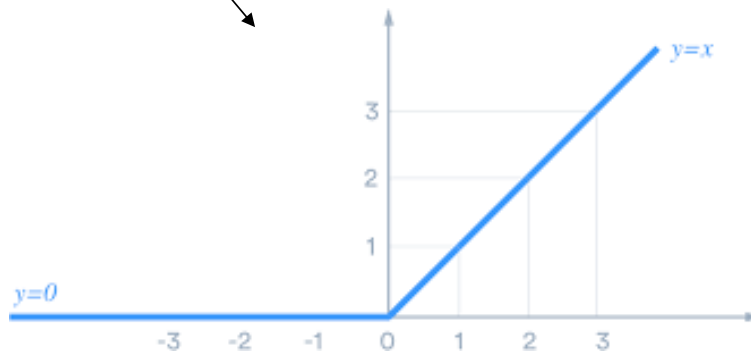
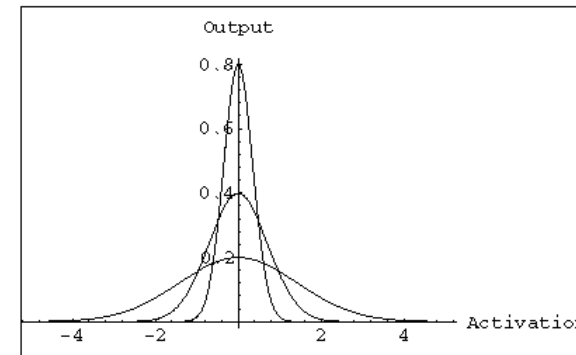
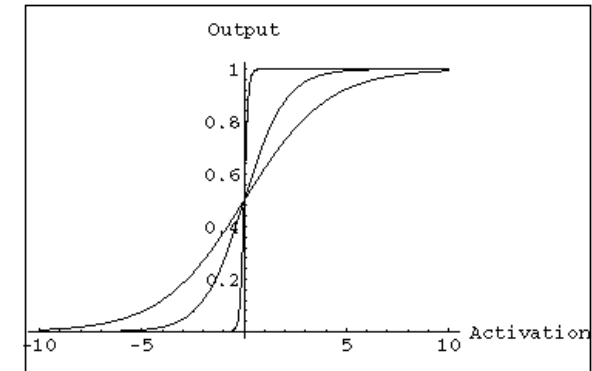
Sigmoid/Logistic

Linear

Hyperbolic tangent
(Tanh)

Gaussian

ReLU (Rectified linear)



Network topologies

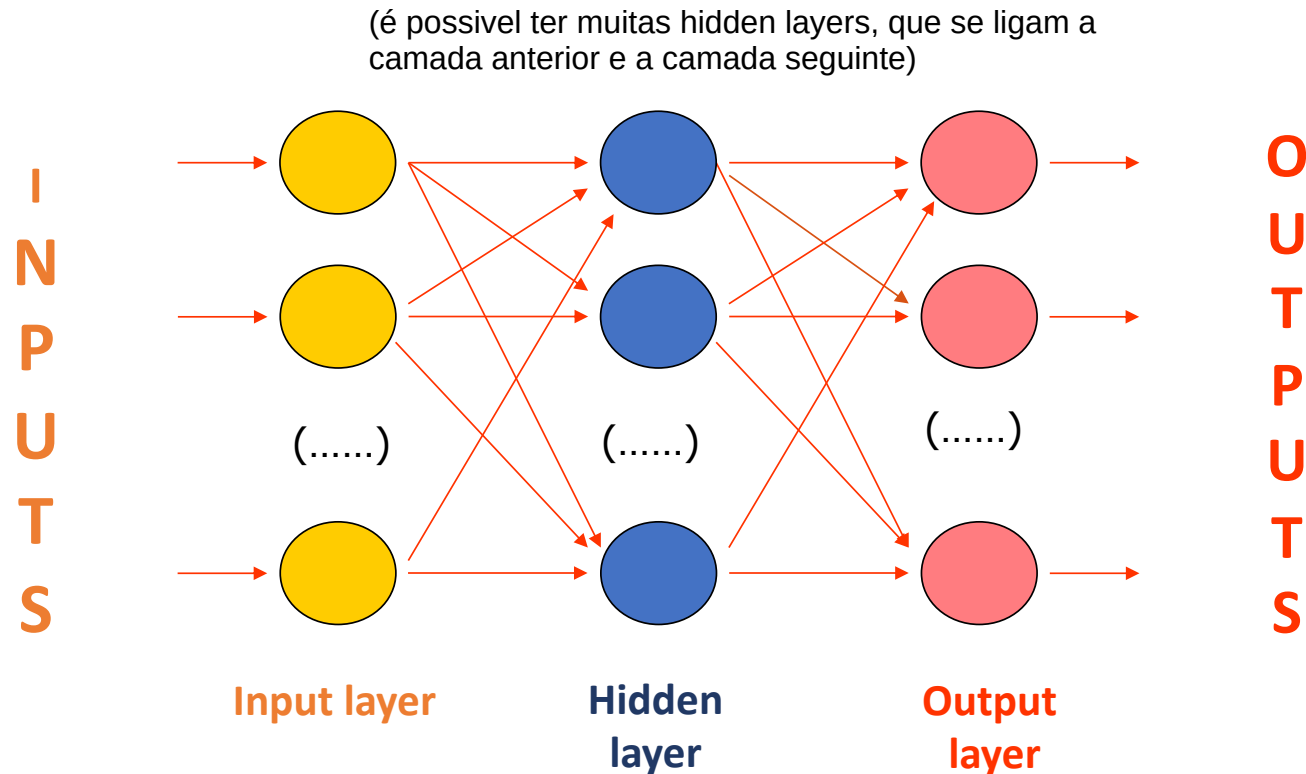
Architecture (or **topology**) - the way nodes interconnect in a network structure (graph)

There are countless types of architectures, each with their own potentialities, falling into two categories: supervised and unsupervised, regarding the way they are trained

Feedforward neural network

(todas estas funções têm de ser deriváveis e devem ser
facéis de derivar (somas, funções quadráticas, logaritmos,
etc.))

(rede neuronal)



Multilayer perceptrons (MLPs)

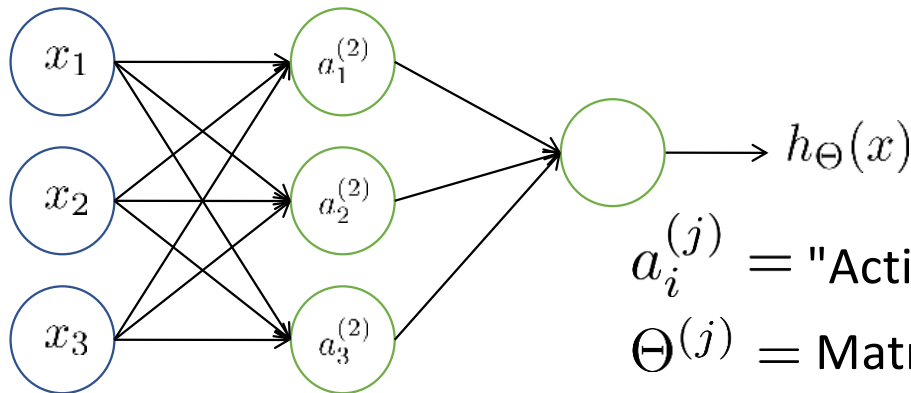
Networks with multiple hidden layers are recently called ***Deep Neural Networks***

(cada seta laranja é uma ligação que tem um peso

quer se determinar a derivada da função de erro, chegar ao output e ver o valor. tem de se derivar a função de erro em função da função de ativação e somatório (?)

o output é calculado com um conjunto de funções compostas uma a seguir à outra)

Representation of a neural network



$a_i^{(j)}$ = "Activation" of neuron i in the layer j

$\Theta^{(j)}$ = Matrix of weights between neurons of layers j and $j + 1$ (rows – destination, columns - origin)

sigmoid =1

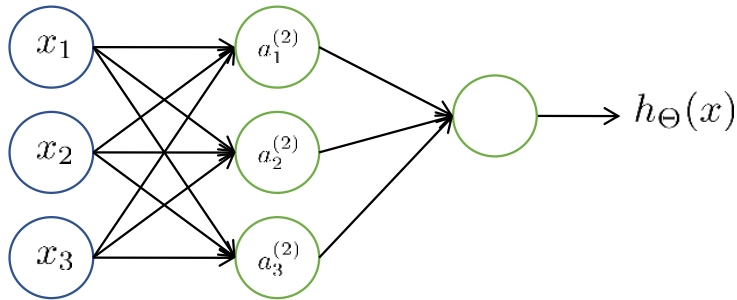
$$a_1^{(2)} = g(\underbrace{\Theta_{10}^{(1)}}_{\text{sigmoid}} x_0 + \underbrace{\Theta_{11}^{(1)}}_{=1} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\underbrace{\Theta_{10}^{(2)}}_{=1} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

Computing the output



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

(conjunto de pesos que se põe numa matriz)

pode por se a correr numa biblioteca do gpu)

$$z^{(2)} = \Theta^{(1)} x$$

(primeiro calcula-se os pesos vezes o vetor do x e depois aplica se a todas as camadas)

$$a^{(2)} = g(z^{(2)})$$

hidden layer activation values

$$\text{add} \quad a_0^{(2)} = 1$$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

$$h_{\Theta}(x) = a^{(3)} = g(z^{(3)}) \longrightarrow \text{output values}$$

Training: supervised learning

Data: Training examples consisting of **inputs** and their desired **outputs**

Objective: To set the values of the **connection weights** that minimize a cost function

Multiple **gradient-descent** algorithms exist:

The most used historically is ***Backpropagation*** and derivatives

Other algorithms: Marquardt-Levenberg, Rprop, Quickprop

Most recent: RMSprop, Adam, SGD, Adagrad (algoritmos muito usados atualmente)

Training: cost function

The cost function of a NN is similar to those used in linear and logistic regression, being the sum of the error function for the set of individual examples

The error function (*loss function*) for each example will typically be given by:

- Sum/ mean of the squared errors (SSE/ MSE), for regression problems
- (Binary) cross entropy (used in logistic regression) for binary classification
- A generalization of the previous one to handle multiple classes

Training: chain rule and computation graphs

As we have already seen, a NN implements the **composition of functions**:

- from the calculation of the activation from the weights and inputs of each neuron
- through the application of the activation functions
- By the interconnection of neurons following the topology of the network

Thus, the cost function of a NN includes many parameters (weights) defined at various levels in the network

Training: chain rule and computation graphs

In order to facilitate the calculation of the derivatives of this function, in relation to the weights to be optimized, the chain rule is used to calculate derivatives of composite functions

Computationally, this allows to organize the process of calculating the output of the NN and the cost function in a computation graph, which can be used for the calculation of derivatives by reversing the direction of calculation and allowing to work layer by layer

Chain Rule

$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$

(regra das derivadas compostas

é aplicada com muitas derivadas parciais)

Backpropagation steps

Forward propagation – calculates the output value(s) for the respective input vector and the error (given the known outputs);

Backpropagation – given the error on that sample, it is propagated back, adjusting the weights of the connections to reduce this error. This process is based on the calculation of the **gradient** of the cost function, using the **chain rule** of partial derivatives for composite functions

***Back-propagation* algorithm**

It is based on the gradient of the error surface that defines the direction of maximum descent (**gradient descent**)

Important parameter: Learning rate- sets the size of the steps in that direction

A sequence of these movements leads to a **minimum** (desirably global)

The training takes place over a number of **epochs** (iterations): examples are typically divided into batches (subsets of examples)

Initial network configuration (connection weights) is randomly generated

Stopping criterion: fixed number of epochs, elapsed time or convergence criterion based on a subset of validation examples

Interpreting the outputs in classification

When using neural nets (and other functional models) to address multiclass classification datasets, we need to convert the output of the model (numerical value) into the discrete value (predicted class) that is desired.

As mentioned we will typically use one neuron per class, which implies to apply **one-hot encoding** to the output variable

In one-hot encoding, there are M output neurons (1 per class), being chosen for any case the class with the highest value.

Using the **softmax** activation function, we can get probabilities for all classes

DNNs for the different supervised tasks

When using neural nets to address supervised learning tasks, three main types of cases arise, which demand different configurations of the network and the training process:

Problem type	Output layer	Activation function (output)	Loss function
Binary classification	One single node	Sigmoid	Binary Cross entropy
Multiclass classification	One node per class	Softmax (sigmoid in each node, normalized to sum 1)	Generalized cross entropy
Regression	One node	Linear (or RELU if only positive outputs are possible)	Mean Squared Error

(se se quiser usar redes neuronais na pratica, temos de saber o que os define. Esta tabela mostra 3 problemas mais comuns)

(ex de overfitting -> quando se estuda demais para um teste)

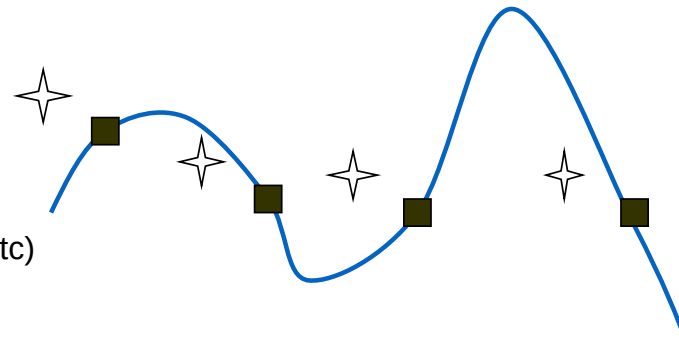
Overfitting and generalization

To train the network many epochs (training iterations) can prevent generalization and promote *overfitting*

It is preferable to stop the training in these cases – **early stopping**; error is measured in a validation dataset; if the validation error improves for a number of consecutive epochs, the training is stopped

The probability of overfitting increases if:

- Few training cases
- Many weights (complexity)



(o overfitting existe em redes neuronais como em todos os modelos para evitar o overfitting, pode se fazer o early stopping, regularização, dropout, etc)

(dropout - treinar a rede neuronal de forma a cada lote ficar numa posição. Assim, os neuronios nao se podem especializar muito, tem de ser mais genericos)

Overfitting and generalization

In DL, overfitting has been addressed using various techniques, such as **regularization** (L1 or L2, similar to linear/logistic regression, called **decay** in NNs), or explicit control of model **capacity** (e.g. number of hidden layers and number of neurons in each)

A specific technique developed for DL models is **dropout**: in each iteration of the training the output of some randomly selected neurons is put to zero. The proportion of zeroed neurons in each iteration is a parameter defined by each layer where dropout applies

Hyperparameter optimization in DNNs

As with all supervised learning approaches, DL models should be the subject of a hyper-parameter optimization process to ensure their best performance. In this case, the most relevant hyper-parameters include:

- Number of layers
- Number of neurons in each layer
- Activation functions to use
- Training algorithm and its parameters (e.g. learning rate)
- Use of dropout and its rate
- Use of regularization and associated parameters
- Using early stopping or not; related parameters

(hiperparametros em redes neuronais -> por exemplo numero de camadas, qual o numero de neuronios em cada camada, etc.

sao hiperparametros porque nao se aprende no algoritmo de treino, tem de se definir antes)

Implementing supervised ML pipelines in Python: feature selection



Regarding feature selection, the following classes are the most relevant:

- **VarianceThreshold**: allows to select the features that have more variability (above a defined threshold) – unsupervised filter
- **SelectKBest, SelectPercentile**: allow to select a number (or fraction) of the features by a ranking mechanism (typically a statistical test performed over each input variable, comparing to the output) – supervised filters
- **RFE**: allows to run a wrapper (recursive feature elimination) selecting the best features working well with a classifier

https://scikit-learn.org/stable/modules/feature_selection.html

Implementing supervised ML pipelines in Python: hyperparameter optimization



Regarding hyperparameter optimization there are two main classes:

- **GridSearchCV**: performs a grid search by testing all combinations of the parameters and values passed as argument (as a dictionary)
- **RandomizedSearchCV**: performs a randomized search where only part of the combinations are tested (the number of combinations is passed as an argument)

https://scikit-learn.org/stable/modules/grid_search.html

Implementation of NNs in Python: keras and tensorflow



To implement python DL models we will use the ***tensorflow*** package: this allows you to create, train, and apply several distinct DL models

Keras (tensorflow.keras) contains interfaces that allow you to use the tensorflow backend; allows you to run on CPU or GPU depending on the machine's h/w

To install tensorflow typically running ***pip install tensorflow*** will be enough

An alternative is to run on google collab:
<https://colab.research.google.com/>

MNIST dataset



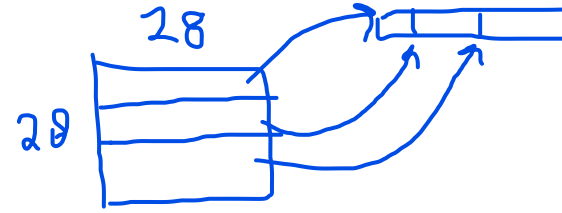
MNIST dataset

Dataset with images of digits in 28x28 pixels grid with 784 features representing pixels in grey scale 0 to 255. Output: class representing the digit (10 classes, digits 0-9). Available as a keras dataset. 60K samples – training set; 10K images – test

This dataset will be used here in the following examples, vectorizing images to a 1D vector with 784 values as inputs for each image, and defining a multi-class classification problem with 10 possible outputs

Jupyter Notebook:
s3_4_dnn_mnist.ipynb

Example: MNIST dataset



```
from tensorflow.keras.datasets import mnist
(train_images, train_labels), (test_images, test_labels) =
    mnist.load_data()
print(train_images.shape, test_images.shape)
print(len(train_labels), len(test_labels))
```

Load data

Check dimensions

```
from tensorflow.keras.utils import to_categorical
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype('float32') / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype('float32') / 255
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
```

Adjust dimensions to make inputs into 1D vector for each example

Standardize values dividing by 255

Convert outputs into categorical variables

DNNs for MNIST



Defining network topology; feed forward

```
from tensorflow.keras import models
from tensorflow.keras import layers
```

```
network = models.Sequential()
network.add(layers.Dense(256, activation='relu'))
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
network.add(layers.Dense(10, activation='softmax'))
```

(errado!!!
a penultima
linha devia
estar em
antepenultim
o e vice
versa)

porque que se tem 10 neuronios porque se tem 10 classes e softmax porque se tem mais de 2 classes

Two hidden layers with 512 and 256 neurons (RelU) and one output layer with 10 neurons (*softmax* - 1 neuron w/ sigmoid function for each class; one-hot encoding)

Exercise

- Play with the number of layers and with the number of neurons in each layer

DNNs for MNIST



```
network.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	401920
dense_1 (Dense)	(None, 256)	131328
dense_2 (Dense)	(None, 10)	2570

```
=====
Total params: 535818 (2.04 MB)
Trainable params: 535818 (2.04 MB)
Non-trainable params: 0 (0.00 Byte)
=====
```

DNNs for MNIST



(numero de vezes que o algoritmo vai ver cada exemplo)

```
network.compile(optimizer='rmsprop',  
                loss='categorical_crossentropy',  
                metrics=['accuracy'])
```

Define training algorithm (RMSprop),
loss function (*cross entropy*) and
error metric (*accuracy*)

Train the model (*fit*):

Define size of each batch and number of iterations (epochs)

```
network.fit(train_images, train_labels, epochs=5, batch_size=128)
```

DNNs for MNIST



(mesmo do sikik learn para prever as probabilidades)

Predicting probabilities ...

```
test_preds = network.predict(test_images)
```

... and class labels

```
test_classes = np.argmax(network.predict(test_images), axis=-1)
```

(vai dar qual o valor de output que é mais elevado)

Evaluate the model in the test set:
calculate loss and accuracy

```
test_loss, test_acc = network.evaluate(test_images, test_labels)
```

Boston housing dataset - regression



Boston
dataset

Predicting the average price of houses in Boston (in the 1970s)
Inputs: several features from the house
404 training examples
102 test examples

```
from tensorflow.keras.datasets import boston_housing
(train_data, train_targets), (test_data, test_targets) =
    boston_housing.load_data()
```

```
print(train_data.shape)
print(test_data.shape)
print(max(train_targets), min(train_targets))
```

(standardização dos dados)

Jupyter Notebook:
s3_4_dnn_housing.ipynb



Boston housing: preparing the data

```
mean = train_data.mean(axis=0)
train_data -= mean
std = train_data.std(axis=0)
train_data /= std
test_data -= mean
test_data /= std
```

Standardizing inputs



Boston housing: DNN

(cria-se um modelo com 2 camadas intermediarias)

```
def build_model(hidden = 64):  
    model = models.Sequential()  
    model.add(layers.Dense(hidden, activation='relu', input_shape=(train_data.shape[1],)))  
    model.add(layers.Dense(hidden, activation='relu'))  
    model.add(layers.Dense(1))  
    model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])  
    return model
```

mean square error

Function to create the model

Two hidden layers with 64 neurons (default); ReLU

Output layer with one output; linear activation

Loss function: MSE

Metric: Mean Absolute Error



Boston housing: DNN

Build the model with training data

```
model = build_model()
model.fit(train_data, train_targets, epochs=80, batch_size=16, verbose=1)

test_mse_score, test_mae_score = model.evaluate(test_data, test_targets)
print(test_mse_score, test_mae_score)
```

Evaluate in test set

Exercises

- try with different number of hidden nodes
- adapt the function to be able to receive other inputs (number layers, metric, dropout, ...)

Biological sequences

- The nucleotide/amino acid sequences constitute a compact 1D representation of complex DNA and RNA/protein molecules
- In many scenarios, it is useful to be able to classify or predict properties/activities of biological molecules (DNA, RNA, proteins) based on their 1D sequence
- The representation typically used for biological sequences is not suitable as a direct input to the learning algorithms we have been using

Representation of sequence datasets

Typically, each example we want to classify will correspond to a distinct sequence of nucleotides or amino acids.

In most cases, sequences can have distinct lengths

Variable(s) to be predicted can be binary, multi-class or numerical

(a diferença é que estas sequencias
nao sao muito faceis de dizer qual a
classe que se vai usar)

Id	Sequence	Output variable
Seq 1	MVSSWYEVSSWKLSV...	0
Seq 2	MYKKEKSTWYESWMKL...	1
Seq 3	MEVSSLSVYVSLAKLSVA...	0
Seq 4	MSYEGSWMTLSVYWNS...	1
Seq 5	MKKLSVLLSVEYPKMSLV...	1
...	...	...

(quando se tem uma sequencia de proteínas e se quer classificar a sequencia (por ex., se tem propriedades antimicrobianas, se é de transporte ou não, qual a sua localização celular, etc.) e pode prever se a sequencia de aminoacidos)

Feature generation: columns

- Previous structure is typically not suitable for traditional learning algorithms as we do not have a defined set of features
- One hypothesis is to consider the symbols in each position as features (one for each column), which brings some problems:
 - Sequences are not **aligned**; aligning can be computationally heavy and make no sense if the sequences are not homologous
 - Sequences have **different sizes** so we will have to: (i) fill the shorter ones with a distinct symbol to put them with the same size; and/or (ii) cut part of the sequences (at the beginning, in the middle or at the end)
 - At each position (column), there may be 4 or 20 hypotheses (depending on the type of sequence) without a defined/logical order – possible solution is to do **one-hot encoding** in each position which greatly increases the number of features

Feature generation: descriptors

- An alternative is to try to represent the sequence by a set of variables, typically binary or numeric, that represent the sequence in a global way, and may correspond to calculated properties of the molecule or to metrics calculated directly from the sequence
- These variables are called **descriptors**, that used as attributes allow you to transform the problem into a typical instance of tabular data learning

Id	Sequence	Output
Seq 1	MVSSWYEVSSWKKLSV...	0
Seq 2	MYKKEKSTWYESWMKL...	1
Seq 3	MEVSSLSVYVSLAKLSVA...	0
Seq 4	MSYEGSWMTLSVYWNS...	1
Seq 5	MKKLSVLLSVEYPKMSLV...	1
...	...	...



Id	Desc 1	Desc 2	Desc 3	...	Output
Seq 1	2.31	3	0		0
Seq 2	1.25	4	1		1
Seq 3	0.98	0	1		0
Seq 4	1.03	5	1		1
Seq 5	2.01	7	0		1
...					...

(coloca se esta tabela como input mas gera se uma matriz tabelar com um conjunto de features)

Feature generation: descriptors

Some examples of descriptors applicable to both DNA/RNA and proteins:

- ***Relative frequency of each symbol*** (Amino acids or nucleotides) – example: for each DNA sequence we will have 4 features of this type that will represent the relative frequencies of each nucleotide in the sequence
- ***Relative frequency of k-mers or k-peptides*** – Example: for $k=2$, and for each amino acid sequence, we will have 400 features that represent the frequency of each dipeptide in the sequence; These attributes can be generated for multiple K values, although for very high values, the number of attributes becomes unbearable.
- ***Cumulative frequencies*** – represent relative frequencies up to certain positions in the sequence – example considering descriptors for 25%, 50%, 75% and 100% of their size, in a sequence of amino acids we would have $4 \times 20 = 80$ descriptors

Note that none of these descriptors takes into account the overall order of the characters in the sequence

Feature generation : protein descriptors

(tambem se podem gerar descritores a partir de propriedades mais fisico quimicas, como solubilidade, carga, peso molecular, etc.)

Some examples of descriptors applicable to **proteins**:

- **Composition** on amino acids, dipeptides and tripeptides; number of each type of bonds, atoms of C, H, N, O, S in the total of AAs
- **Physico-chemical properties** of the protein: e.g. charge, molecular weight, aromaticity, isoelectric point, *instability index*, etc
- **2D structure**: number of helices, sheets, etc.
- **Auto-correlations** of different AA properties (Moreau-Broto, Moran, Geary)
- **Aminoacid Pseudo-composition**

Classification/prediction of properties from sequences

Id	Sequence	Output
Seq 1	MVSSWYEVSSWKKLSV...	0
Seq 2	MYKKEKSTWYESWMKL...	1
Seq 3	MEVSSLSVYVSLAKLSVA...	0
Seq 4	MSYEGSWMTLSVYWNS...	1
Seq 5	MKKLSVLLSVEYPKMSLV...	1
...	...	...



Id	Desc 1	Desc 2	Desc 3	...	Output
Seq 1	2.31	3	0		0
Seq 2	1.25	4	1		1
Seq 3	0.98	0	1		0
Seq 4	1.03	5	1		1
Seq 5	2.01	7	0		1
...	...	...	...	...	...

Step 1:

Choose encoding and type of descriptors to use
Generate table with descriptors and output variable



Best model + predicted error

Step 2:

Set data partitions for error estimation/model selection
Run "usual" learning pipeline



Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

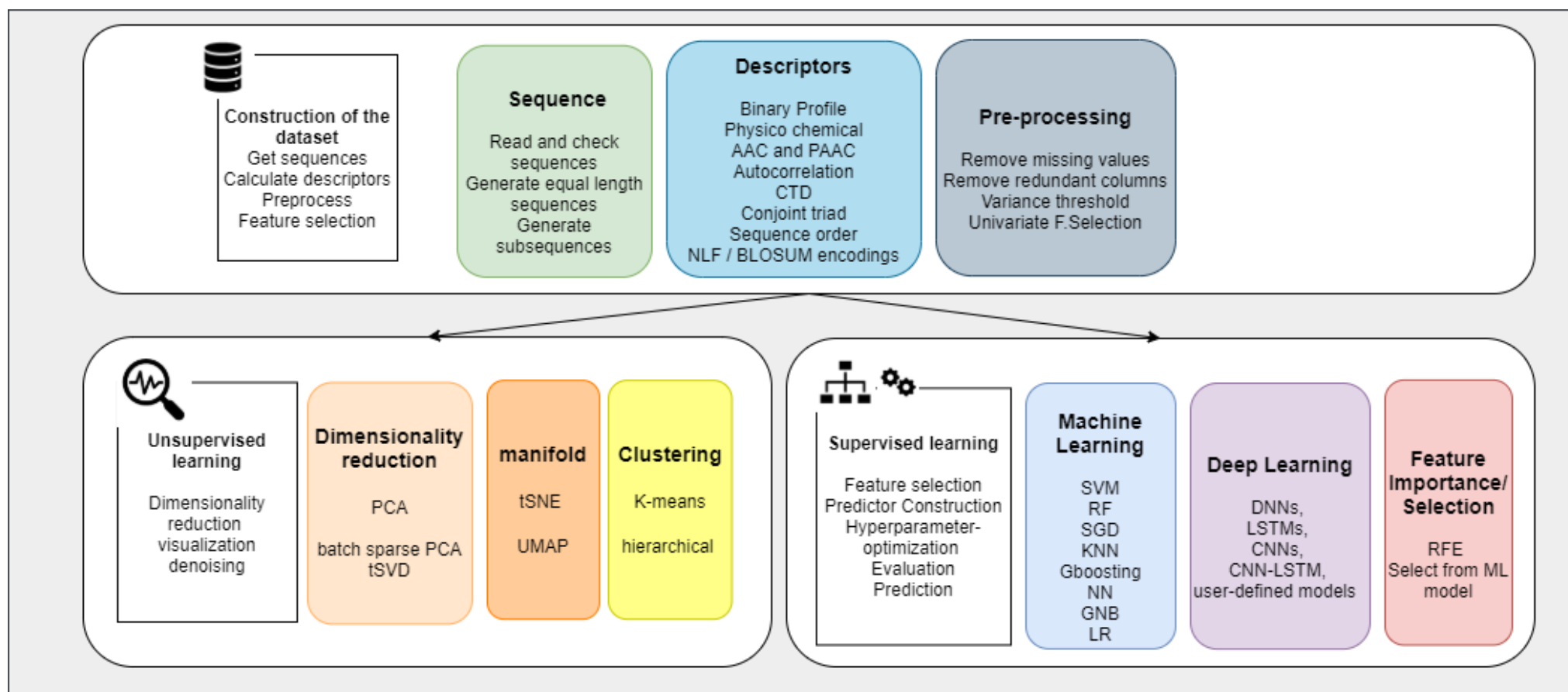
ProPythia: A Python package for protein classification based on machine and deep learning

Ana Marta Sequeira^{a,*}, Diana Lousa^b, Miguel Rocha^a

This process can be implemented using different Python packages for the two steps
The ProPythia package, developed at U. Minho, allows you to do both in a single package

<https://github.com/BioSystemsUM/propythia>

ProPythia package



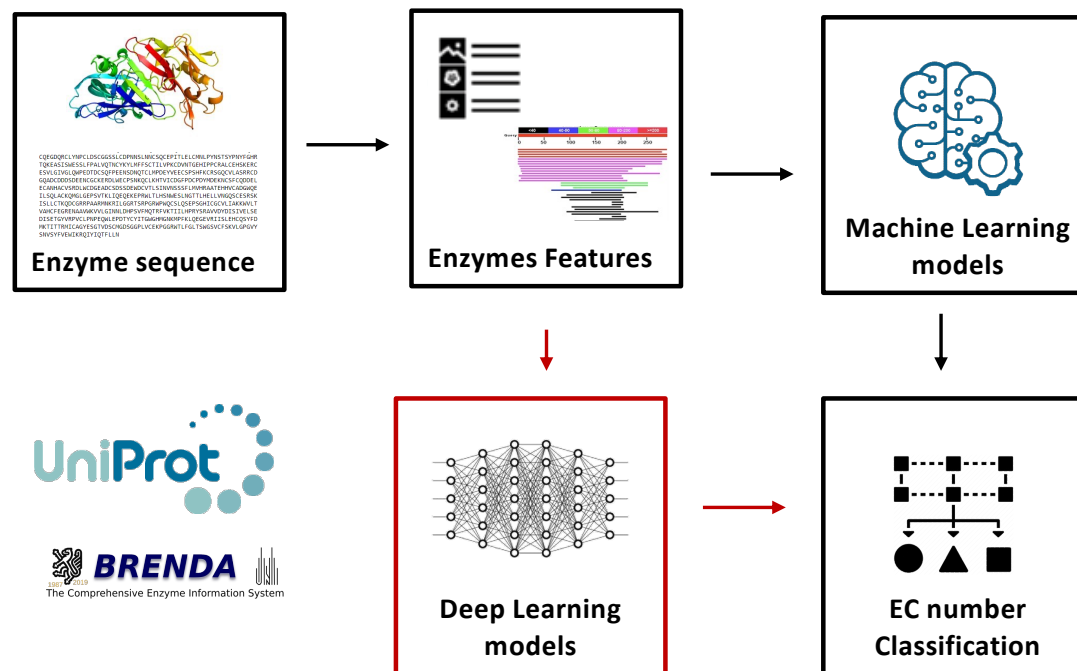
Bioinformatics tools that use ML/DL

- WoLF PSORT predicts the subcellular localization of proteins in eukaryotes using a k-nearest neighbor (k-NN) classifier, which is a type of machine learning algorithm, to make its predictions
- LocTree3: employs a hierarchical approach that integrates multiple localization predictors, some of which may use AI-based methods
- TMHMM: A tool that uses a hidden Markov model to predict transmembrane helices in proteins
- MEMSAT: A tool that predicts transmembrane helices and the topology of integral membrane proteins using an artificial neural network

Classification/prediction of properties from sequences - examples



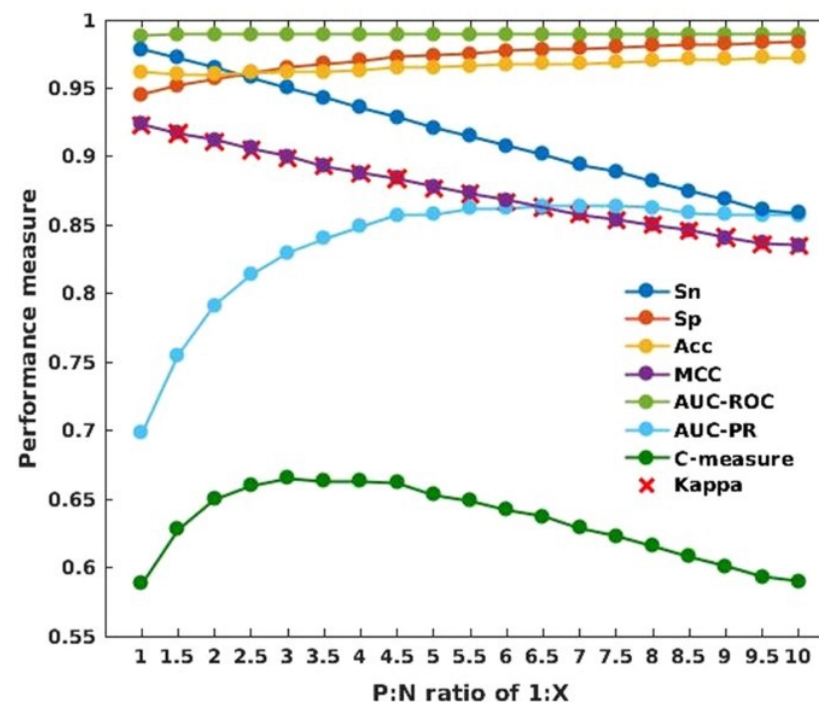
Prediction of enzyme classes (EC number) from amino acids



Classification/prediction of properties from sequences - examples



Prediction of antimicrobial peptides



Article | [Open Access](#) | [Published: 26 January 2018](#)

AmPEP: Sequence-based prediction of antimicrobial peptides using distribution patterns of amino acid properties and random forest

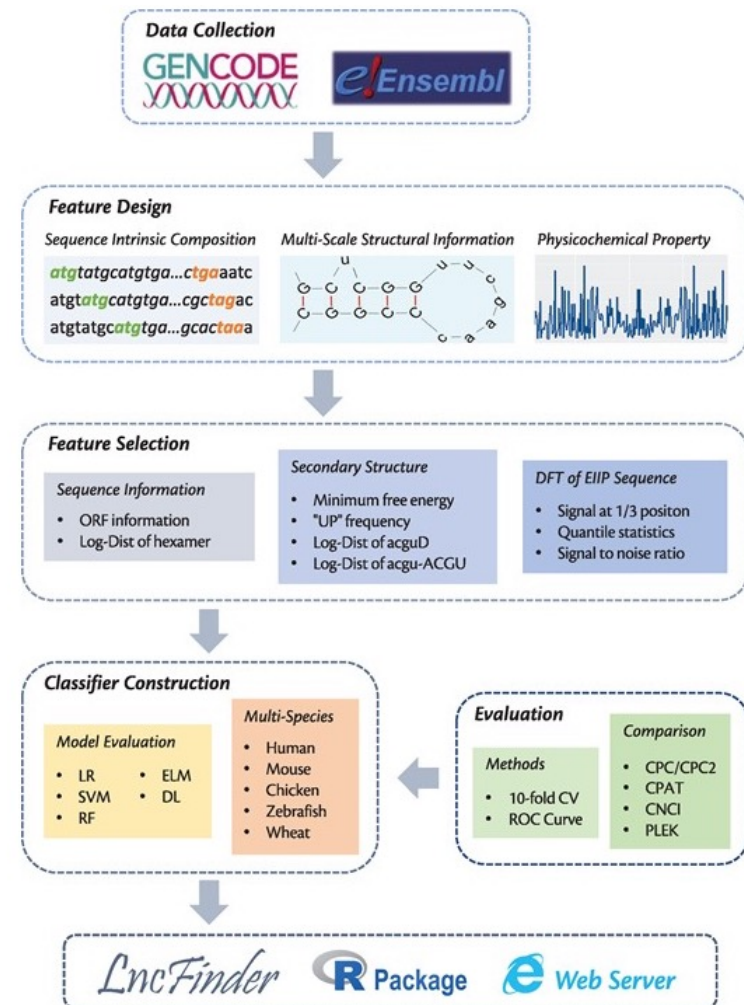
[Pratiti Bhadra](#), [Jielu Yan](#), [Jinyan Li](#), [Simon Fong](#) & [Shirley W. I. Siu](#)

[Scientific Reports](#) **8**, Article number: 1697 (2018) | [Cite this article](#)

Classification/prediction of properties from sequences - examples



Identifying long non coding RNAs

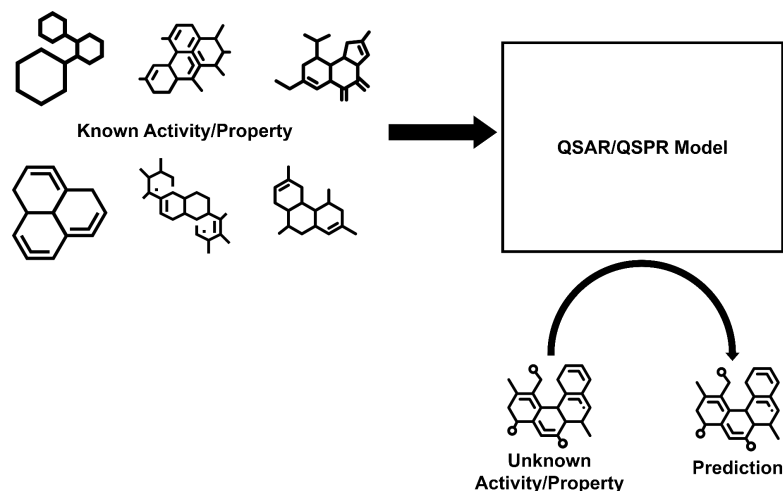


Introduction to QSAR/QSPR

(machine learning para descoberta de farmacos

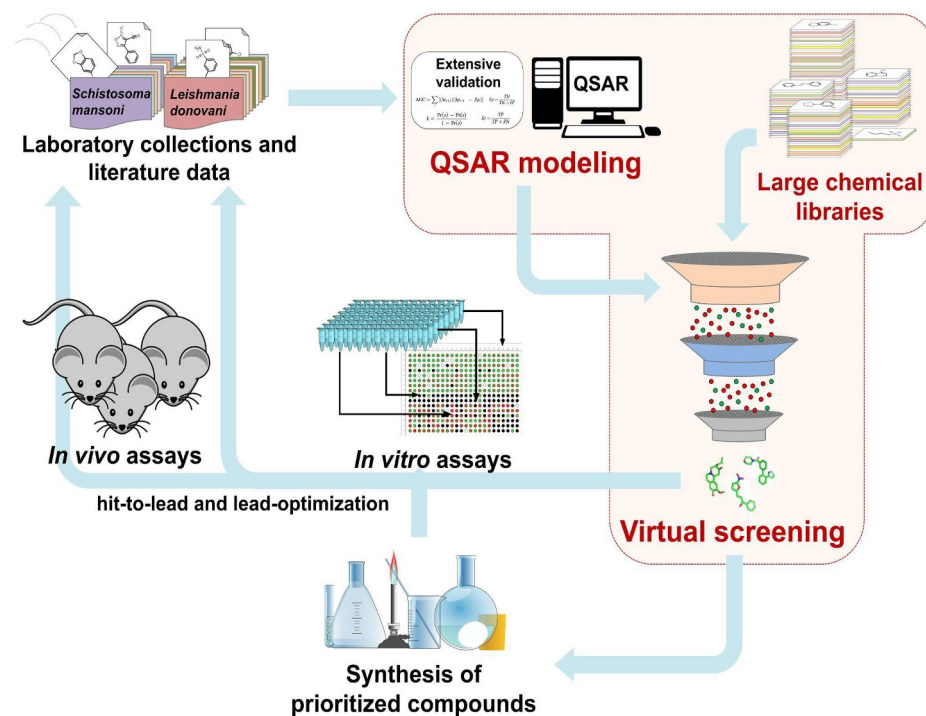
é uma area bastante antiga)

- **Quantitative Structure-Activity/Property Relationship (QSAR/QSPR)** are computational modelling approaches to **predict the relationships between molecular structures and biological/chemical activities or properties**;
 - QSAR → predict drug potency, toxicity, bioaccumulation, etc; (atividade biológica)
 - QSPR → predict solubility, melting point, surface tension, etc; (propriedades químicas como solubilidade, melting point, etc.)
- The basic assumption for QSAR/QSPR models is that **similar molecules have similar activities/properties**.



Need for QSAR/QSPR Models

- **Experimental methods** were used to determine molecular activities/properties, but they are **time-consuming and expensive**;
- Predictive models **leverage machine learning** to accelerate research by making accurate predictions based on existing data;
- This allows researchers to focus resources on the **most promising candidates**.

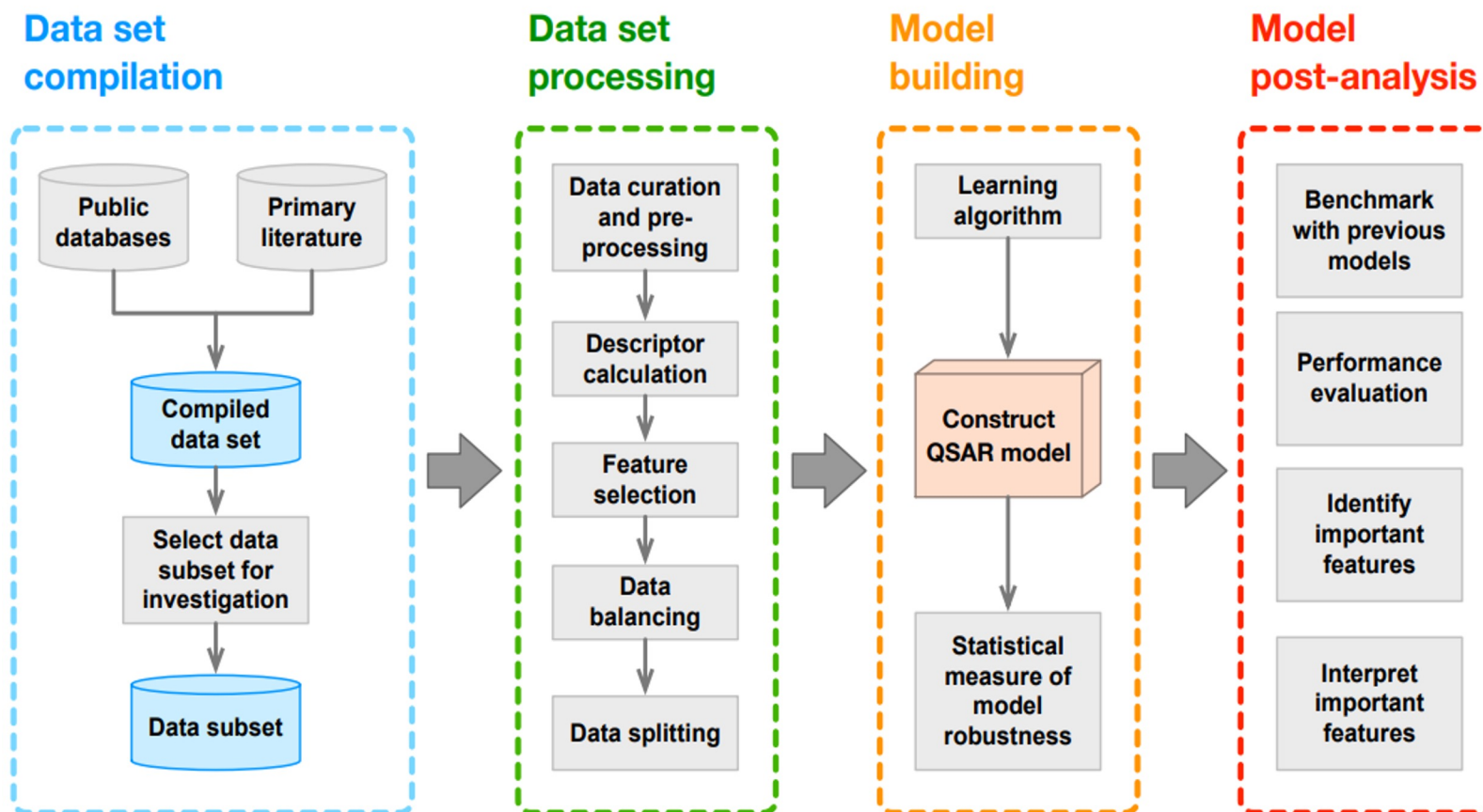


Neves BJ, Braga RC, Melo-Filho CC, Moreira-Filho JT, Muratov EN and Andrade CH (2018) QSAR-Based Virtual Screening: Advances and Applications in Drug Discovery. Front. Pharmacol. 9:1275. doi: 10.3389/fphar.2018.01275

QSAR/QSPR Application Fields

- **Drug Discovery:** Predicting the effectiveness as a drug candidate;
- **Food and Flavor Chemistry:** Predicting the flavor and aroma properties;
- **Environmental Chemistry:** Predicting the toxicity and environmental impact;
- **Materials Science:** Designing new materials with desired properties like conductivity or strength.

QSAR/QSPR Workflow



Nantasenamat C. (2020) Best Practices for Constructing Reproducible QSAR Models. In: Roy K. (eds) Ecotoxicological QSARs. Methods in Pharmacology and Toxicology. Humana, New York, NY.

juntar os datasets

modelo de machine
learning

Compound dataset representation

Typically, each example we want to classify will correspond to a distinct compound that can be represented in one of the previous ways

Variable(s) to be predicted can be binary(s), multi-class or numeric(s)

Id	compound	Output
Comp 1	CH ₃ -CH=O	2.1
Comp 2	CH ₃ -CH(CH ₃)-CH ₂ -OH	3.5
Comp 3	C ₁ =CC=CC=C ₁	0.4
Comp 4	C[C@@H](C(=O)O)N	3.1
Comp 5	CC(=O)NCCC1=CNc2c1cc(OC)cc2	1.3
...	...	...

(o output pode ser binario, numerico, etc.)

Feature generation: descriptors

- An alternative is to represent the compound by a set of variables, typically binary or numerical, which may correspond to known properties of the compound or calculated by chemoinformatics software, or to local structural patterns present or not in the compound
- We name these variables as **descriptors (or fingerprints)**, that used as features allow you to turn the problem into a typical instance of tabular datasets (os descritores podem ser usados em diferentes formas fingerprints - subestruturas das estruturas)

Id	Compound	Output
Comp 1	CH ₃ -CH=O	2.1
Comp 2	CH ₃ -CH(CH ₃)-CH ₂ -OH	3.5
Comp 3	C ₁ =CC=CC=C ₁	0.4
Comp 4	C[C@@H](C(=O)O)N	3.1
Comp 5	CC(=O)NCCC1=CNc2c1cc(OC)cc2	1.3
...	...	...



Id	Desc 1	Desc 2	Desc 3	...	Output
Comp 1	2.31	3.01	0		2.1
Comp 2	1.25	4.24	1		3.5
Comp 3	0.98	0.35	1		0.4
Comp 4	1.03	5.01	1		3.1
Comp 5	2.01	7.23	0		1.3
...					...

QSAR/ QSPR supervised ML pipeline

Id	Sequência	Output
Comp 1	CH3-CH=O	2.1
Comp 2	CH3-CH(CH3)-CH2-OH	3.5
Comp 3	C1=CC=CC=C1	0.4
Comp 4	C[C@@H](C(=O)O)N	3.1
Comp 5	CC(=O)NCCC1=CNC2c1cc(OC)cc2	1.3
...	...	...



Id	Desc 1	Desc 2	Desc 3	...	Output
Comp 1	2.31	3.01	0		2.1
Comp 2	1.25	4.24	1		3.5
Comp 3	0.98	0.35	1		0.4
Comp 4	1.03	5.01	1		3.1
Comp 5	2.01	7.23	0		1.3
...					...

Step 1:

Choose representation and type of descriptors to use
Generate table with descriptors and output variable



Step 2:

Define data partitions for error estimation/model selection
Run "usual" learning pipeline

Best Model + Predicted Error

This process can be implemented using different Python packages for the two steps

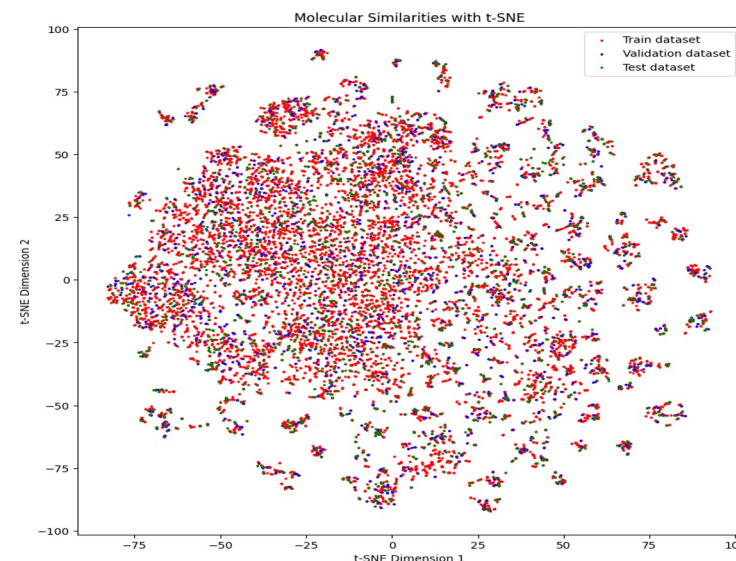
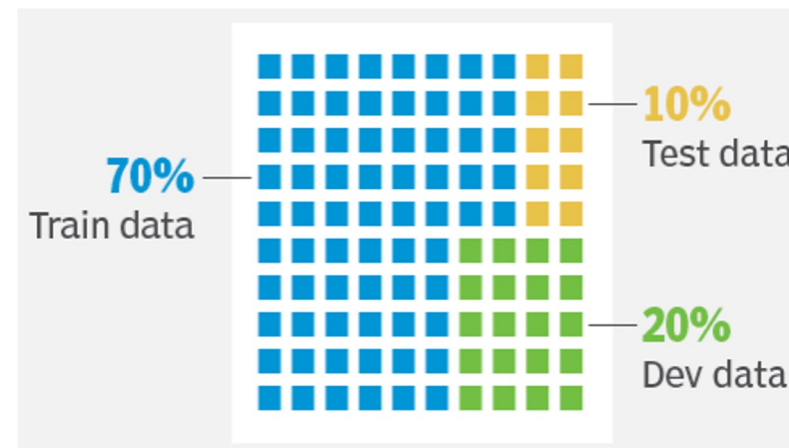
The DeepMol package, developed at U. Minho, allows you to do both in a single package

<https://github.com/BioSystemsUM/deepmol>

Data Splitting

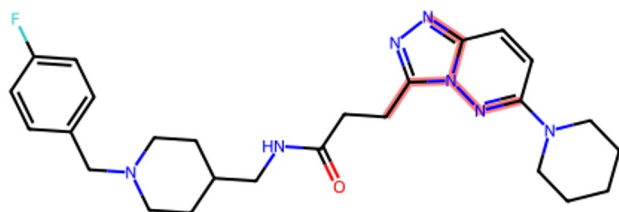
- Important in ML to assess a **model's performance on unseen data**, enabling the evaluation of generalization capabilities and preventing overfitting;
- For chemical data it is common to perform data separation based on **molecular structures or similarities**:
 - **uniform data split**, seeking to maintain a balanced distribution of molecular structures or similarities across sets;
 - **separation of highly similar molecules** from less similar ones across sets.
- For single and multitask classification tasks it is also important to maintain **stratification of classes**.

(separação de dados de treino (para treinar os modelos), dados para testes (idealmente usar apenas uma vez, ou seja, validar apenas o modelo em dados onde nunca se viu nada) e o dev data serve para ver se os 3 sets estão bem espalhados pelo dataset ou não pode tentar se evitar ter coisas parecidas no teste e no treino: estabelecer um limite de similaridade (ex: se se tiver proteínas e se quiserem testar, convém não serem muito próximas))

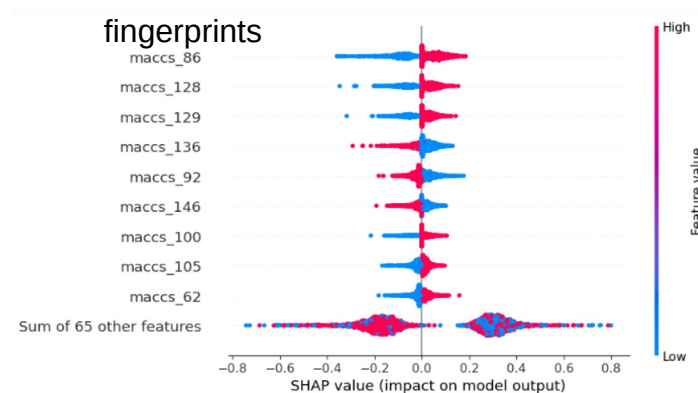


Model Interpretability

- Building accurate models is essential, but understanding **why and how the model makes predictions** is equally important;
- Model interpretation provides insights into the **relationships between molecular features and properties/activities**;
- Feature Importance techniques help to identify which molecular descriptors have the **most impact on predictions**.



(os maccs permitem classificar o composto como toxico e nao toxico, adoçante e nao adoçante, etc.)



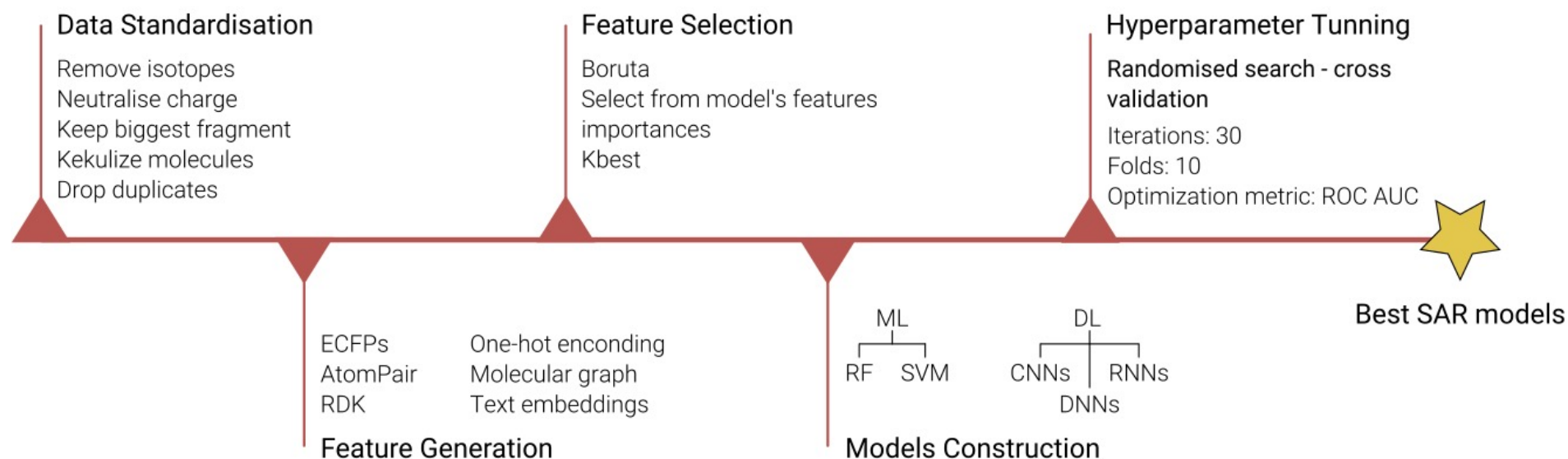
DeepMol

- Our in-house **Machine and Deep Learning Framework for Computational Chemistry**;
- Freely available on GitHub at:
<https://github.com/BioSystemsUM/DeepMol>
- Documentation and examples:
<https://deepmol.readthedocs.io/en/latest/>
- Provides a Pipeline module that can **automate an end-to-end ML pipeline** optimizing each step and respective hyperparameters.



DeepMol

Classification/Prediction of Properties from Compounds - DeepMol

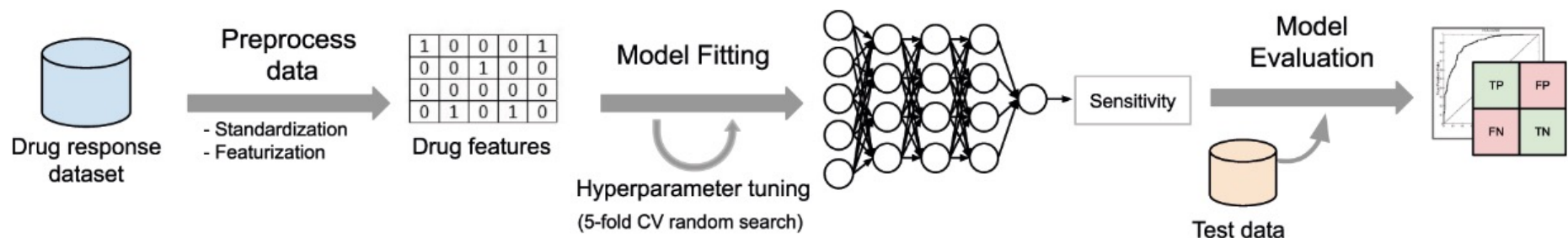


DeepMol python package
<https://github.com/BioSystemsUM/DeepMol>

Classification/prediction of properties from compounds - examples

Prediction of drug sensitivity of cancer cell lines:

Includes classification issues (sensitive vs non sensitive) and regression(IC50)



DeepMol-based implementation

Evaluating molecular representations in machine learning models for drug response prediction and interpretability

Delora Baptista , João Correia, Bruno Pereira and Miguel Rocha

From the journal *Journal of Integrative Bioinformatics*

<https://doi.org/10.1515/jib-2022-0006>

exemplos de coisas publicadas